Project Title: A Reconfigurable, RISC-like Processing Element for Multi-Precision Neural Network Inference

1. Project Goal:

To define, design, implement, and test, a highly efficient, instruction-set-driven Processing Element (PE) for accelerating quantized neural network inference. The core objective is to explore the architectural trade-offs between performance, power, and area by supporting multiple integer precisions (INT8 and INT4). Students will be tasked with making and justifying key design decisions, resulting in a custom accelerator architecture optimized for edge computing scenarios.

2. Core Hardware and Baseline Instruction Set Specifications:

- **Architecture:** RISC-like, controlled by an 8-bit op-code.
- **Data Interface:** Two 32-bit input operand busses (operand_A_in, operand_B_in), one 8-bit output bus (result_out).
- **Core Compute Units:** Four reconfigurable 8x8 signed integer multipliers.
- **Primary Register:** One 64-bit signed accumulator.
- **Arithmetic:** All operations use signed integer arithmetic unless specified otherwise.

The following compute operations form the baseline for the PE. The final instruction set will be expanded based on the student's design choices.

- MAC8: operand_A_in and operand_B_in are treated as four packed INT8 numbers each. Perform four INT8*INT8=>INT16 multiplies and add the four 16-bit results to the 64-bit accumulator.

  $$\sum_{1}^{N} input_i \times weight_i \Rightarrow \text{accumulator}$$

- MAC4: similar to MAC8, but works on INT4 operands. See next section for details.
- ADD_BIAS: operand_A_in is a 32-bit bias value. Add this value to the accumulator.

  $$bias + accumulator \Rightarrow \text{accumulator}$$

- RESCALE: Rescales the 64-bit signed accumulator into an INT8 or INT4 activation_input. See next section for details.

  $$((accumulator * M) >> N) \Rightarrow \text{activation_input}$$

- ACTIVATE: Applies the chosen activation function on the internally stored activation_input. The final INT8 result is placed on result_out. See next section for details.

  $$f_{activation}(activation_input) \Rightarrow result_out$$

3. Architectural Exploration Points:

A key component of this project is for students to explore the design space and make justified decisions on the following points:

A. **MAC4 Performance vs. Complexity Trade-off:**
   The PE must support INT8 MAC operations. For INT4, students must analyze and choose one of the following two implementation strategies:
   a. **Energy-Saving Mode:** Reuse the 8x8 multipliers to perform 4x4 operations. This approach saves memory bandwidth and energy but does not increase throughput (four MACs per cycle).
   b. **High-Throughput Mode:** Reconfigure the four 8x8 multipliers to function as eight parallel 4x4 multipliers. This approach doubles throughput but increases the complexity and area of the multiplier datapath.

B. **RESCALE Operation Implementation:**
   The RESCALE operation ((accumulator * M) >> N) requires a 32x32 multiplication. A full single-cycle 32x32 multiplier is too large. Students must design an efficient alternative by choosing one of the following paths:
   a. **Internal Multi-Cycle Unit:** Design a single RESCALE instruction that triggers an internal state machine to perform the multiplication over several clock cycles, reusing the existing 8x8 multipliers, and ending with the required shift_right operation.
   b. **RISC-like Staged Instructions:** Define a sequence of new, simpler op-codes that perform partial-product calculations. The external controller would be responsible for issuing this sequence of instructions to complete a full 32x32 multiplication (e.g., RESCALE_STEP_1, RESCALE_STEP_2, etc.). The last op-code will perform the required shift_right operation.

C. **Activation Function Flexibility:**
   Students must design the hardware for the activation function. The exploration should cover:
   a. **Implementation Choice:** Choose between a hard-coded ROM LUT, a flexible RAM-based LUT, or synthesizable combinational logic (e.g., for ReLU).
   b. **Instruction Set Definition:** Based on the choice, define the necessary op-codes. A RAM-based LUT, for example, requires a LOAD_LUT instruction. A design supporting multiple hard-coded functions may require an ACTIVATE instruction with an activation_type operand.

D. **Custom Instruction Set Definition:**
   Beyond the core compute operations, students must define a minimal and efficient set of instructions for setup, control, and testing. This includes operations for clearing the accumulator, managing data flow, and providing debug visibility into the PE's internal state (e.g., reading the accumulator value).

4. Scope of Work:

A. Architecture Definition:

- Explore the architectural choices detailed in section 3. Learn how architecture definition decisions affect performance, power, complexity, and area.
- Learn how four INT8xINT8 multipliers can be used for four, or eight, INT4xINT4 multipliers. And how they can be used to implement an INT32xINT32 multiplier.
- While defining the Custom Instruction Set, think about what is needed for testing the PE and for consistent operation. You may need to return to this stage later in the design cycle.
- Specify architectural level verification.

B. RTL Design (Verilog/VHDL):

- Implement the data-path and control logic to support the architecture definition.
- If a multi-cycle unit is used for the re-scale operation, an FSM should be designed to control the multi-cycle operation.
- Specify unit testing and use it throughout the RTL Design phase.

C. Rigorous Verification (partially concurrent with Architecture Definition, and RTL Design):

- Understand the different kinds of verification, and their special needs: functional, timing, and production testing.
- Develop a rigorous test plan that includes architectural level verification and unit testing. If needed, define special PE logic and instructions to aid testing.
- Use unit level testing, in simulation, during the RTL design of the different units.
- Use architectural level testing, in simulation, once the entire PE is designed.
- Develop a comprehensive testbench.
- Verify the output of the PE, doing a MAC, rescaling, and activation, against a golden reference (e.g., a Python/NumPy model performing the same operation with the same precision). Verify both INT8 and INT4 operations.

D. ASIC Synthesis and Optimization Analysis:

- **Target Technology:** Synthesize the RTL code to a standard cell library.
- **Optimization Focus:** Within the defined architecture optimize for performance and power consumption. Analyse longest timing paths and learn how small sacrifices in power and area can greatly improve performance. Analyse power consumption and learn how you can improve it by small sacrifices in area and complexity.
- **Report & Analysis:** Generate and analyze detailed synthesis reports (clock frequency, area, power, total latency, throughput).
- **Trade-off Discussion:** Discuss how the design choices impact the speed,

area, and power.

5. Key Challenges and Learning Outcomes:

- **CPU Architecture:** Gaining practical experience with RISC-like principles and instruction set design.
- **VLSI Design:** Designing reconfigurable datapaths, managing trade-offs between area, power, and clock speed. Practical experience with synthesis tools for area, power, and timing.
- **NN Acceleration:** Understanding the hardware implementation of core quantization concepts like multi-precision arithmetic and per-tensor scaling.
- **System-Level Thinking:** Analyzing how hardware architecture choices impact software (control complexity) and overall system performance.

6. Deliverables:

- **Comprehensive Project Report:** Detailing architectural choices, justification for each exploration point (MAC4, RESCALE, Activation), the final instruction set, verification strategy, and a detailed analysis of synthesis results (area, power, timing).
- **Synthesizable RTL Code (Verilog/VHDL).**
- **Rigorous Verification Testbench** and simulation results demonstrating the functionality of all instructions and a complete inference pass for a single neuron.
- **Detailed Synthesis Reports** and a comparative analysis of power and performance for different operational modes (INT8 vs. INT4).